

MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES
(A Constituent unit of MAHE, Manipal)

Cryptology – 7_Lab

Reg. Number	Name	Branch
261100690006	MOHAMMED HARIS	Cybersecurity



MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

Write Python code to perform both encryption and decryption)

a. columnar transposition

```
def encrypt(text, key):
    columns = len(key)
    result = ""

    for number in sorted(key):
        column = key.index(number)

        for position in range(column, len(text), columns):
            result += text[position]

    return result


def decrypt(ciphertext, key):
    columns = len(key)
    text_length = len(ciphertext)

    column_lengths = []
    for column in range(columns):
        length = 0
        for position in range(column, text_length, columns):
            length += 1
        column_lengths.append(length)

    column_text = [""] * columns
    position = 0

    for number in sorted(key):
        column = key.index(number)
        length = column_lengths[column]
        column_text[column] = ciphertext[position:position + length]
        position += length

    result = ""
    rows = (text_length + columns - 1) // columns

    for row in range(rows):
        for column in range(columns):
            if row < len(column_text[column]):
                result += column_text[column][row]

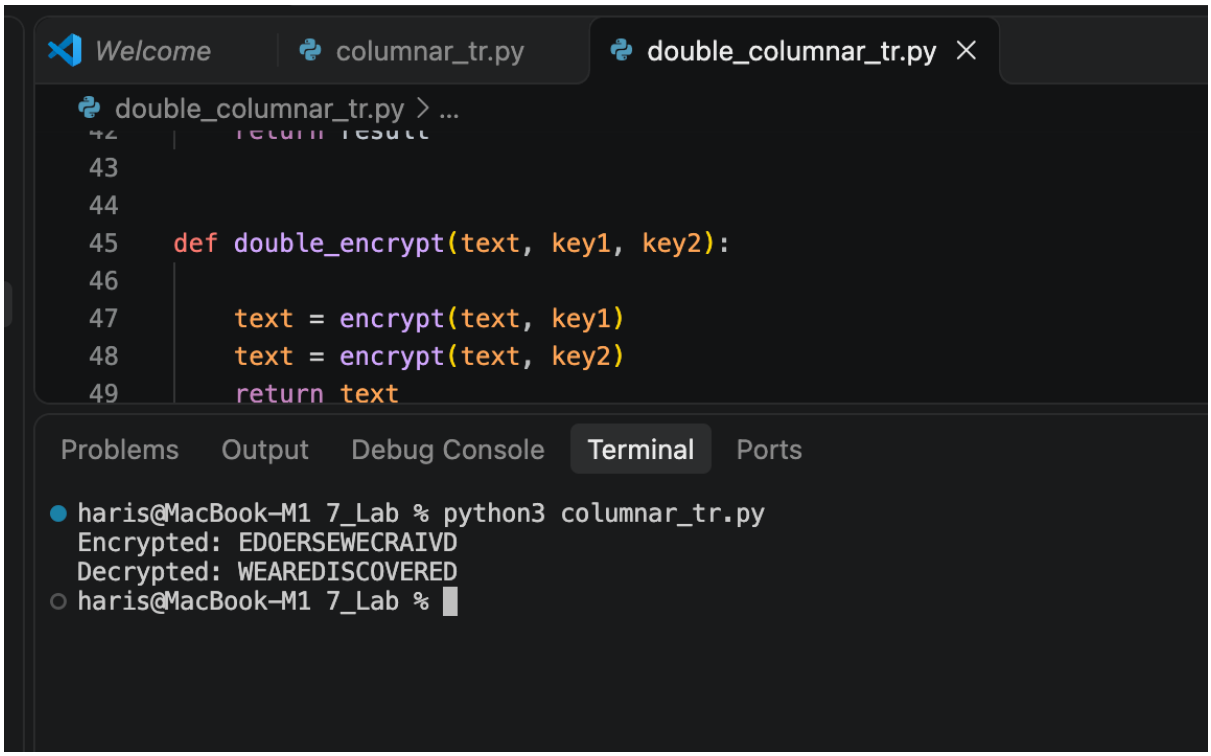
    return result


message = "WEAREDISCOVERED"
key = "3142"

encrypted = encrypt(message, key)
```

```
decrypted = decrypt(encrypted, key)

print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```



b. double columnar transposition

```
def encrypt(text, key):
    columns = len(key)
    result = ""

    for number in sorted(key):
        column = key.index(number)

        for position in range(column, len(text), columns):
            result += text[position]

    return result


def decrypt(ciphertext, key):
    columns = len(key)
    text_length = len(ciphertext)

    column_lengths = []
    for column in range(columns):
        length = 0
```

```
        for position in range(column, text_length, columns):
            length += 1
        column_lengths.append(length)

    column_text = [""] * columns
    position = 0

    for number in sorted(key):
        column = key.index(number)
        length = column_lengths[column]
        column_text[column] = ciphertext[position:position + length]
        position += length

    result = ""
    rows = (text_length + columns - 1) // columns

    for row in range(rows):
        for column in range(columns):
            if row < len(column_text[column]):
                result += column_text[column][row]

    return result


def double_encrypt(text, key1, key2):

    text = encrypt(text, key1)
    text = encrypt(text, key2)
    return text

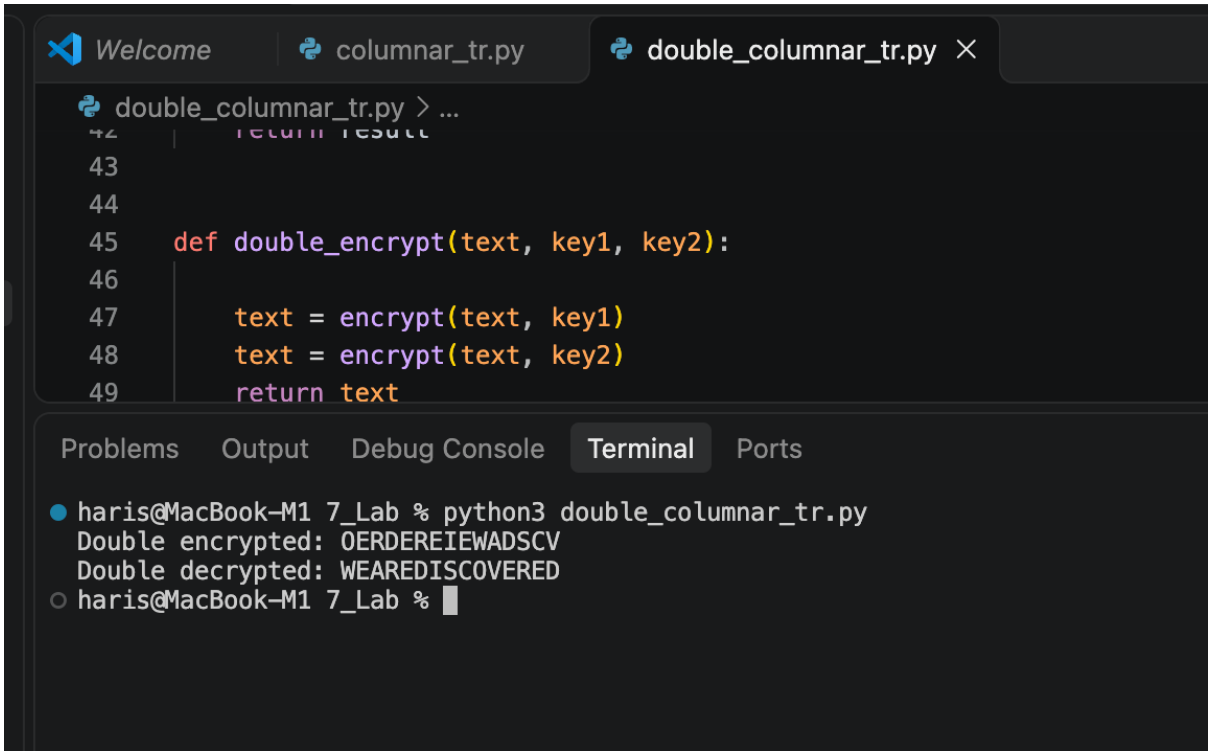

def double_decrypt(ciphertext, key1, key2):

    ciphertext = decrypt(ciphertext, key2)
    ciphertext = decrypt(ciphertext, key1)
    return ciphertext


message = "WEAREDISCOVERED"
key1 = "3142"
key2 = "2413"

encrypted = double_encrypt(message, key1, key2)
decrypted = double_decrypt(encrypted, key1, key2)

print("Double encrypted:", encrypted)
print("Double decrypted:", decrypted)
```



Explore the OpenSSL tool

OpenSSL is an open-source command-line toolkit and library implementing SSL/TLS protocols and general-purpose cryptography. It's the most widely used tool for generating keys, certificates, encrypting/decrypting data, and testing secure connections.

Core capabilities:

Area	What it does
Key generation	Creates RSA, EC, DSA key pairs
Certificates	Creates, signs, and inspects X.509 certificates (used in HTTPS)
Encryption/decryption	Symmetric (AES, etc.) and asymmetric (RSA) encryption of files
Hashing	Computes message digests (SHA-256, MD5, etc.)
Digital signatures	Signs and verifies data for authenticity/integrity
TLS testing	Connects to servers to inspect/test their SSL/TLS setup

Common commands:

```
# Generate an RSA private key
```

```bash
openssl genpkey -algorithm RSA -out private.pem -pkeyopt rsa_keygen_bits:2048

# Extract the public key
openssl pkey -in private.pem -pubout -out public.pem

# View key details
openssl rsa -in private.pem -text -noout

# Hash a file
openssl dgst -sha256 file.txt

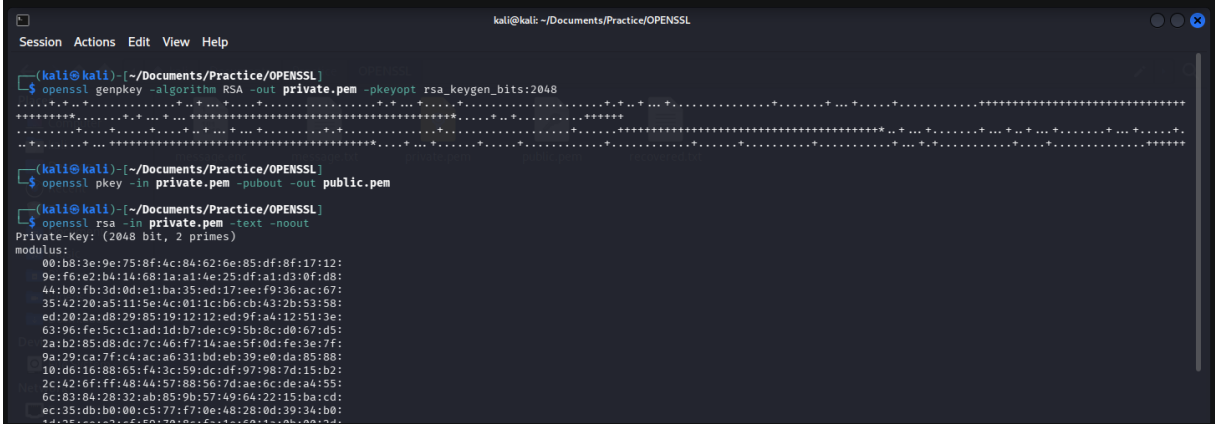
# Encrypt/decrypt a file (symmetric, AES)
openssl enc -aes-256-cbc -salt -in file.txt -out file.enc
openssl enc -aes-256-cbc -d -in file.enc -out file.txt

# Encrypt/decrypt with RSA public/private keys
openssl pkeyutl -encrypt -pubin -inkey public.pem -in msg.txt -out msg.enc
openssl pkeyutl -decrypt -inkey private.pem -in msg.enc -out msg.txt

# Sign and verify data
openssl dgst -sha256 -sign private.pem -out sig.bin file.txt
openssl dgst -sha256 -verify public.pem -signature sig.bin file.txt

# Generate a self-signed certificate
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365

# Test a server's TLS connection
openssl s_client -connect example.com:443
```





OpenSSL provides solid foundation to HTTPS, VPNs, email encryption, and code signing across the internet - most web servers, browsers, and security tools rely on it (or libraries derived from it) to implement TLS/SSL securely.